

Persistent Login

Persistent Login

1. Objective

The objective of this solution is to provide a persistent login experience for B2B users, allowing them to continue their session after the Navitaire authentication token expires without requiring them to manually log in again.

This solution is applicable only when the user explicitly enables Persist Login during login. The proposed solution is designed to work within Navitaire's authentication limitations while minimizing the impact on the existing application architecture.

2. Current Challenge

Navitaire authentication has the following limitations:

- Access token validity is approximately 15 minutes.
- No native Remember Me / Persistent Login capability.
- Token expiry requires complete authentication again.

Because of this limitation, users returning after token expiry must log in again even though they may still be working in the same browser session.

3. Design Principles

- No modification to existing business APIs.
- No authentication validation added to every API.
- No impact to existing authentication flow.
- Minimal database usage.
- Backward compatible.
- Multiple browser sessions supported.
- Persist Login only when the user explicitly enables it.

4. Functional Behaviour

Persist Login is optional. A checkbox will be provided on the B2B login screen:

☒ Keep me signed in during this browser session

If unchecked, the current authentication behaviour remains unchanged. If checked, Persistent Login is enabled.

5. Scope

Applicable	Notes
B2B Portal	Persistent Login applies when the user explicitly enables it during login.
B2C Portal	Persistent Login applies when the user explicitly enables it during login.
Mobile apps	Persistent Login applies when the user explicitly enables it during login.

6. Important Behaviour

Persistence exists only within the current browser session. Persistent Login is removed if the user:

- Closes the browser tab.
- Closes the browser.
- Clears session storage.

 This is **not** a Remember Me implementation across browser restarts.

7. New Components

Area	Components / Enhancements
UI	• Persist Login checkbox • Session Storage • Re-initiate Login handling • Retry failed request • Auth listeners
Middleware	• POST /persistLogin • POST /reInitiateLogin
Database	New Persist Login table

8. Persist Login Flow

Step 1: User Authentication

1. User logs in successfully using username, password, and OTP.
2. Navitaire login succeeds.
3. Authentication token is generated.

Step 2: Persist Login Registration

1. User selects Persist Login.
2. UI invokes `POST /persistLogin`.
3. The following payload is submitted:

```
1 {
2   "username": "agent01",
3   "password": "*****",
4   "domain": "B2B",
5   "persist": true
6 }
```

9. Backend Validation

1. Backend validates the current user token.
2. Backend calls AuthInfo.
3. Authenticated username is retrieved.
4. Authenticated username is compared with payload username.
5. If there is a mismatch, backend returns `403 Forbidden`.

11. Secure Storage

The backend performs the following actions:

- Generate secure UUID.
- Store password without encryption to reduce processing overhead.
- Rely on the request integrity check, server-side validation, expiry enforcement, and HTTPS for protection.
- Store record in the Persist Login database.

Field	Description
Password	Secure random UUID
Username	B2B username
Password	Password value stored without encryption to improve performance;

	protected by integrity validation and server-side controls.
Domain	B2B
Created UTC	Timestamp
Expiry UTC	Created + 12 hours
Last Used UTC	Nullable

12. Response

Backend returns:

```

1 {
2   "uid": "8f3c1d9a-7b24-4e6f-92d8-c5a71e3f4b60",
3   "expiryTime": "23-07-2026T12:20Z"
4 }
```

UI stores the PersistId & expiry time in Session Storage. **No password is stored in the browser.**

13. Business API Execution

- User continues working.
- Existing APIs remain 100% unchanged.
- No database lookup is required.
- No additional authentication is added.
- No performance impact is introduced.

14. Token Expiry

Eventually, Navitaire may return `401 Unauthorized` or `440 Login Timeout`. The current request fails.

15. User Experience

Instead of redirecting to Login, the user sees the following message:

Your session has expired. Click Continue to resume your session.

Continue redirects the user to the Dashboard. The Dashboard automatically performs `POST /reInitiateLogin`.

16. ReInitiate Login Request

UI retrieves the PersistId from Session Storage. Before transmission, UI generates an encoded request token using:

- Persist UUID
- Username
- Request content length

The generated value is Base64 encoded and sent to the backend.

Integrity Check Process

Input

Field	Value
Username	ITTEST
Content Length	2055

1. Generate MD5 hash

```
1 MD5("ITTEST2055")
2 = 517b39d808a132b0dcdded9497c65d6e2
```

2. Generate / attach request UID

```
1 8f3c1d9a-7b24-4e6f-92d8-c5a71e3f4b60
```

3. Combine UID and MD5

```
1 8f3c1d9a-7b24-4e6f-92d8-c5a71e3f4b60
2 +
3 517b39d808a132b0dcdded9497c65d6e2
4 =
5 8f3c1d9a-7b24-4e6f-92d8-c5a71e3f4b60517b39d808a132b0dcdded9497c65d6e2
```

4. Base64 encode

```
1 0GYzYzFkOWEtN2IyNC00ZTZmLTkyZDgtYzVhNzF1M2Y0YjYwNTE3YjM5ZDgwOGExMzJiMGRjZGVkOTQ5N2M2NWQ2ZTI=
```

Backend Validation

1. Receive the following values:

- Username
- Content Length
- UID
- Integrity Token (Base64 string)

2. Recalculate the MD5 hash:

```
1 MD5(username + contentLength)
```

3. Concatenate the UID and recalculated hash:

```
1 UID + recalculated_MD5
```

4. Base64 encode the concatenated value.

5. Compare the encoded value with the received Integrity Token:

- Match →  Integrity Valid
- No match →  Integrity Failed / Potential Tampering

Example:

```
1 | {{{url}}}/reinitateLogin?uid=8f3c1d9a-7b24-4e6f-92d8-c5a71e3f4b60 |
```

This prevents exposing the raw UUID directly over the wire.

17. Backend Processing

1. Decode Base64.
2. Extract UUID.
3. Retrieve Persist Record.
4. Validate that the record exists, is active, and is not expired.
5. Recalculate request validation hash.
6. Compare validation values.
7. If valid, retrieve the stored password.
8. Call Navitaire Login.
9. Receive new token.
10. Update Last Used Time.
11. Return authentication token.

18. UI Processing

1. UI receives authentication token.
2. UI stores the token.
3. UI retries the previously failed API automatically.
4. User continues working with no manual login required.

19. Expired Persist Session

If current UTC is greater than Expiry UTC:

1. Backend returns `401`.
2. Backend deletes the Persist Record.

3. UI removes the PersistId from Session Storage.

4. User is redirected to Login.

20. Security

- Persist Login is optional.
- Password is never returned to UI.
- Username is validated against authenticated AuthInfo.
- Secure random UUID is generated by backend.
- Expiry is enforced server-side.
- Expired sessions are automatically cleaned up.
- Only the same browser session can reuse the Persist ID.
- Session Storage ensures data is cleared when browser/tab session ends.
- All communication is over HTTPS.

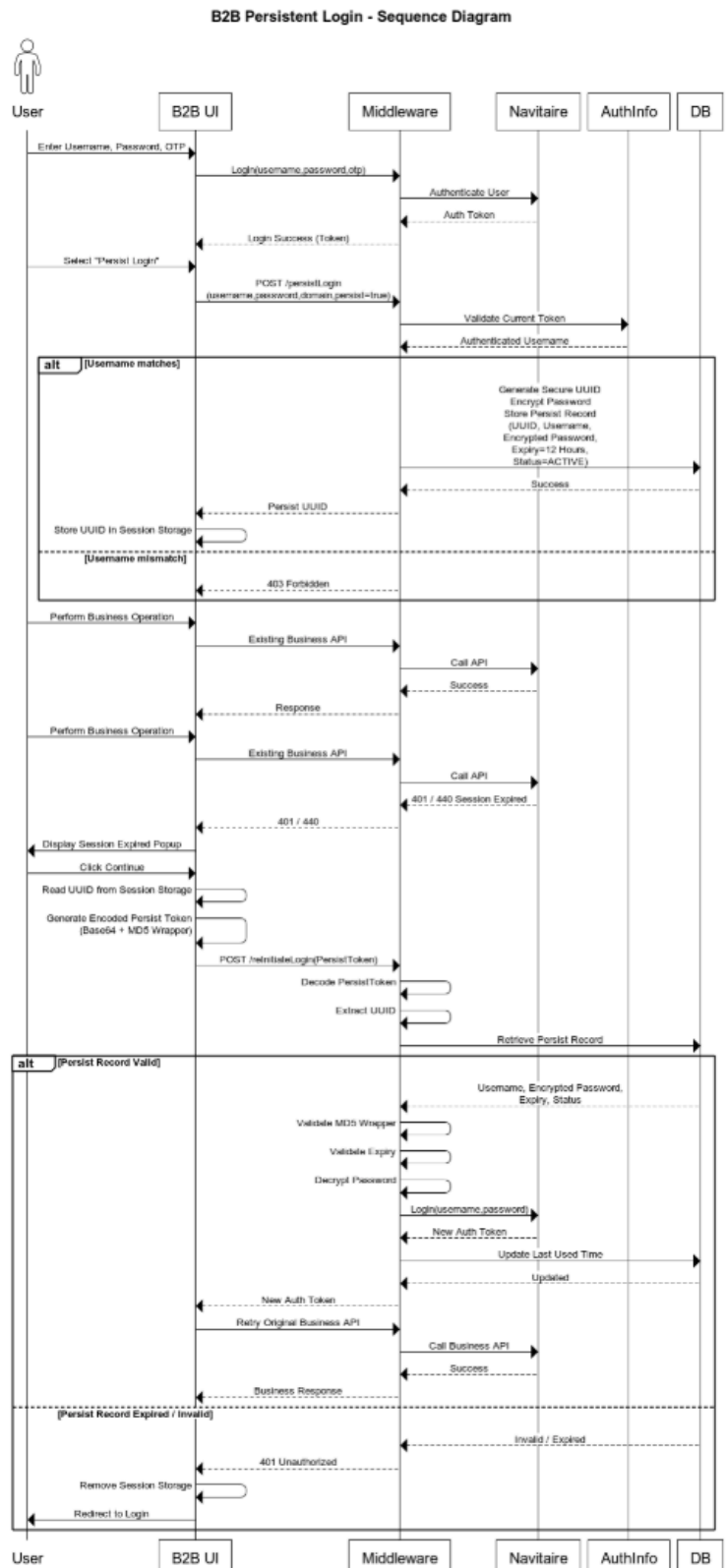
21. Advantages

- Only two new APIs introduced.
- Existing approximately 200 business APIs remain unchanged.
- No API versioning required.
- No database lookup for every request.
- Improved performance by avoiding password encryption/decryption overhead.
- Supports multiple concurrent browser sessions.
- Backward compatible.
- Limited implementation effort.
- Easy rollback if required.

22. Limitations

- Available only within the same browser session.
- Closing the browser or tab removes the persisted session.
- Password is stored without encryption during the configured persistence window to reduce processing overhead; access is controlled through integrity validation, expiry enforcement, and server-side checks.
- Password is securely stored in encrypted form for the configured persistence window.

23. Sequence Diagram



24. Recommendation

This solution provides a lightweight and low-impact implementation of persistent login for the B2B portal. By introducing only two dedicated APIs, `PersistLogin` and `ReInitiateLogin`, it avoids changes to the existing authentication mechanism and approximately 100+ business APIs. This significantly reduces implementation effort, regression risk, and performance overhead.

The design also aligns with the business requirement that persistence is limited to the current browser session while supporting multiple concurrent browser sessions for the same user.